

A COMPLETE PRACTITIONER'S GUIDE

Advanced Retrieval- Augmented Generation

From First Principles to Production-Grade
Agentic AI Systems — with LangChain, LangGraph & Gemini

MODULE TEN

Capstone Project & Deployment

Beginner to Industry Level • Assumes only basic Python

MODULE TEN

Capstone Project with Deployment

This is where everything converges. Over nine modules you learned the components of RAG one at a time; now you assemble them into a single, coherent, production-ready system — and ship it. You will build a complete application with multi-document ingestion, hybrid retrieval and cross-encoder re-ranking, a self-correcting agentic core in LangGraph, a RAGAS evaluation suite, and LangSmith monitoring. Then you will deploy it: wrap it in a FastAPI service, containerise it with Docker, push it to the cloud, and architect it to scale. By the end you will have done the full job — not a notebook demo, but a deployed system.

WHERE WE ARE

Modules 1-9 each taught one part of RAG in isolation. This module has no new theory to teach — instead it shows how the parts fit together into one production system and how to deploy it. Treat this as a worked reference architecture: a project structure, real code, and a deployment path you can adapt to any RAG product. A dedicated **Production RAG** section follows this module with the deep-dive operational topics — caching, cost and token optimisation, security, guardrails, and CI/CD.

10.1 Introduction

Every module so far ended with a working piece of RAG — an ingestion pipeline, a retriever, an agentic graph, an evaluation suite. Each was correct, but each lived alone, in its own script, run by hand. That is not a product. A product is a single, coherent system that a user can actually reach, that runs reliably without you babysitting it, that can be measured and monitored, and that can grow with demand.

This capstone builds exactly that. We will assemble the components from Modules 1-9 into one application — call it the **Enterprise Knowledge Assistant** — and then do the part that most tutorials skip entirely: **deploy it**.

The journey of this module has two halves. The first half is **integration**: taking nine modules' worth of components and wiring them into a clean, well-structured codebase where ingestion feeds retrieval, retrieval feeds the agentic core, the agentic core is

monitored and evaluated. The second half is **deployment**: turning that codebase into a running service the outside world can use — exposing it through an API, packaging it in a container, running it in the cloud, and shaping its architecture so it stays fast and reliable as users multiply.

A note on mindset before we start. Up to now, "it works" meant "the script produced the right answer when I ran it." From this module on, "it works" means something stricter: it works *for users you will never meet, under load you did not anticipate, while you are asleep*. That shift — from code that runs to a system that *operates* — is what this module, and the Production RAG section after it, is really teaching.

10.2 Why This Topic Matters

WHY THIS MATTERS

There is a well-known, painful pattern in applied AI: a team builds an impressive RAG prototype in a notebook, demos it, everyone is excited — and then it never ships, because turning a notebook into a deployed, reliable, scalable service is a different and harder skill that nobody on the team has. This module is that skill. The ability to take a RAG system from "works on my machine" to "running in production, serving real users" is what makes an AI engineer genuinely valuable — and it is exactly what the capstone-style question "walk me through how you would deploy this" probes in senior interviews.

Three concrete reasons this module repays close study:

1. **Integration reveals what isolated learning hides.** Components that each worked alone can clash when combined — shared configuration, consistent data flow, startup ordering. Building the whole system surfaces and resolves those issues.
2. **Deployment is a distinct discipline.** APIs, containers, cloud, scaling, monitoring — none of this is RAG-specific, all of it is essential, and it is where prototypes go to die. Learning it here, on a system you understand end to end, is the right way to learn it.
3. **A finished, deployed project is your proof.** For your portfolio, your interviews, and your own confidence, "I built and deployed a production agentic RAG system" is worth more than any number of notebooks.

10.3 Capstone Overview: What We Are Building

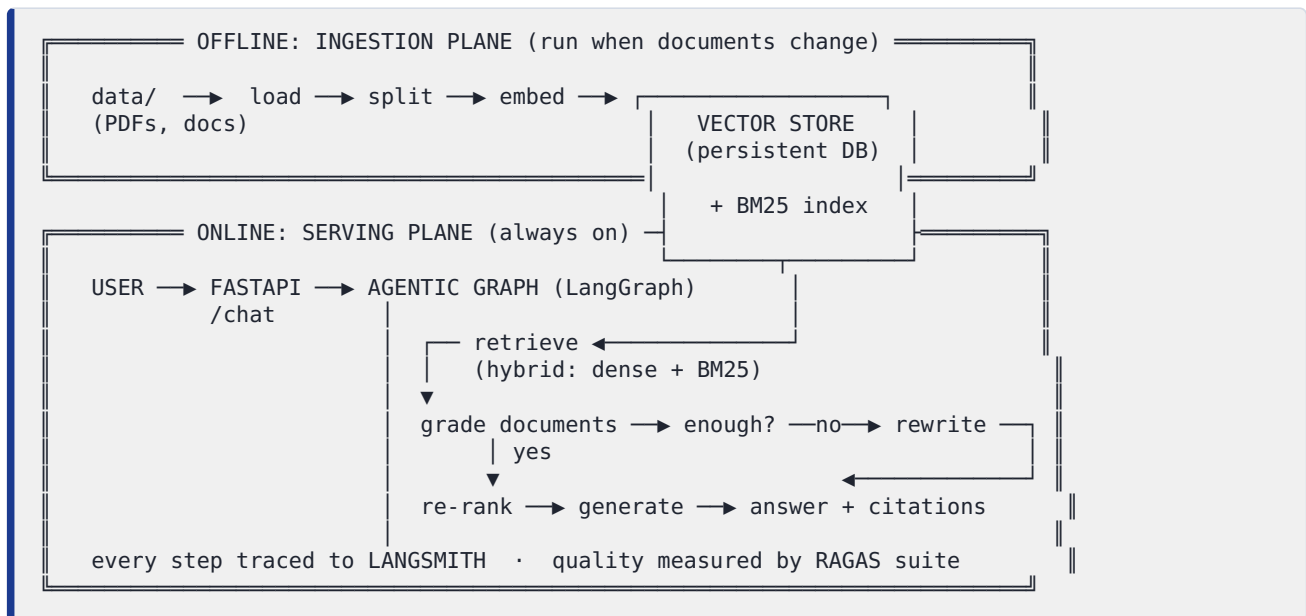
The **Enterprise Knowledge Assistant** is a deployed service that lets users ask natural-language questions over a private collection of documents and receive accurate, grounded, cited answers. It is the synthesis of the whole book. Each capability traces directly to an earlier module:

Capability	What it does	From module(s)
Multi-document ingestion	Loads a folder of mixed documents, chunks, embeds, stores them	2, 3, 4
Hybrid retrieval	Combines dense (semantic) and sparse (BM25) search	5
Cross-encoder re-ranking	Two-stage retrieve-then-rerank for precise context	6
Agentic RAG core	A self-correcting LangGraph workflow: grade, retry, generate	7, 8
Conversation memory	Remembers earlier turns within a session	8
Evaluation suite	RAGAS metrics scoring the system's quality	9
Monitoring	LangSmith tracing of every request	10 (this module)
API service	A FastAPI endpoint the outside world can call	10
Containerisation & cloud	Docker image, deployed and scalable	10

This is, deliberately, a real system — the kind a company would actually run. And because its architecture is generic, it doubles as a template for the other projects on the book's project list: a *PDF Chatbot* is this system with one document type; a *Customer Support AI* is this system over help-centre articles; a *Research Assistant* is this system with a web-search tool added to the agent. Build this one well and you have built the pattern for all of them.

10.4 System Architecture

A production RAG system has two distinct planes, and keeping them separate is the first architectural decision. The **offline ingestion plane** runs occasionally (when documents change) and builds the knowledge base. The **online serving plane** runs continuously and answers user requests. They share only the vector store.



Three architectural principles are visible here, and all three were foreshadowed in earlier modules:

- **Ingestion and serving are separate.** Ingestion is a batch job; serving is a long-running service. They must not be tangled — re-embedding documents inside a user request would be disastrously slow.
- **The agentic graph is the brain.** The FastAPI layer is thin — it receives a request and hands it to the LangGraph application (Module 8). All the intelligence lives in the graph.
- **Observability wraps everything.** Every request is traced (LangSmith) and the system's quality is continuously measurable (RAGAS). A production system you cannot see into is a production system you cannot trust.

10.5 Project Folder Structure

A clean structure is not cosmetic — it is what makes a multi-component system maintainable. Here is the capstone's layout:

```

enterprise-rag/
├── .env                # secrets: API keys (never committed)
├── .gitignore          # ignores .env, __pycache__, chroma_db/, etc.
├── requirements.txt    # pinned Python dependencies
├── Dockerfile          # container image definition
├── docker-compose.yml  # local multi-service orchestration
├── data/              # source documents to ingest
├── chroma_db/         # the persistent vector store (generated)
├── tests/
│   └── test_set.json  # evaluation questions + ground truth
├── src/
│   ├── config.py      # central configuration & env loading
│   ├── ingest.py      # OFFLINE: build the knowledge base
│   ├── retriever.py   # hybrid retrieval + re-ranking
│   ├── graph.py       # the agentic RAG core (LangGraph)
│   ├── evaluate.py    # RAGAS evaluation suite
│   └── app.py         # ONLINE: the FastAPI service

```

Notice the structure mirrors the architecture. `ingest.py` is the offline plane. `app.py` is the online plane. `retriever.py` and `graph.py` are the shared logic. `config.py` is the single place all settings live — so a change to the model or chunk size happens in one file, not scattered across many. `evaluate.py` is the quality instrument from Module 9.

10.6 Component 1 — Configuration and Multi-Document Ingestion

Two foundations first: a central configuration file, and the ingestion pipeline that builds the knowledge base.

```

# src/config.py — one place for every setting
import os
from dotenv import load_dotenv

load_dotenv()

GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY")
LANGSMITH_KEY = os.getenv("LANGCHAIN_API_KEY")

DATA_DIR = "data"
CHROMA_DIR = "chroma_db"
COLLECTION_NAME = "enterprise_knowledge"

EMBEDDING_MODEL = "models/text-embedding-004"
LLM_MODEL = "gemini-2.0-flash"
RERANKER_MODEL = "cross-encoder/ms-marco-MiniLM-L-6-v2"

CHUNK_SIZE = 1000
CHUNK_OVERLAP = 150
RETRIEVE_K = 20 # broad candidate set for stage 1
RERANK_TOP_N = 4 # precise final set after re-ranking
MAX_RETRIES = 2 # agentic self-correction loop guard

```

```

# src/ingest.py — OFFLINE: build the knowledge base (Modules 2-4)
from datetime import date
from pathlib import Path
from langchain_community.document_loaders import DirectoryLoader, PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_chroma import Chroma
from src import config

def build_knowledge_base():
    """Load → validate → split → enrich → embed → store. Run when docs change."""
    # 1. LOAD every PDF in the data folder (Module 2)
    loader = DirectoryLoader(config.DATA_DIR, glob="**/*.pdf",
                             loader_cls=PyPDFLoader, show_progress=True)
    documents = loader.load()

    # 2. VALIDATE — drop pages whose text extraction failed (Module 2)
    documents = [d for d in documents if d.page_content.strip()]
    print(f"[ingest] {len(documents)} non-empty pages loaded.")

    # 3. SPLIT into chunks (Module 2)
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=config.CHUNK_SIZE, chunk_overlap=config.CHUNK_OVERLAP)
    chunks = splitter.split_documents(documents)

    # 4. ENRICH metadata + assign DETERMINISTIC IDs (Modules 2, 4)
    ids = []
    for i, chunk in enumerate(chunks):
        src = Path(chunk.metadata.get("source", "unknown")).name
        chunk.metadata["file_name"] = src
        chunk.metadata["ingested_on"] = str(date.today())
        ids.append(f"{src}::chunk_{i}")

    # 5. EMBED + STORE in a persistent Chroma vector store (Modules 3, 4)
    embeddings = GoogleGenerativeAIEmbeddings(model=config.EMBEDDING_MODEL)
    store = Chroma(collection_name=config.COLLECTION_NAME,
                    embedding_function=embeddings,
                    persist_directory=config.CHROMA_DIR)
    store.add_documents(chunks, ids=ids) # deterministic IDs → no duplicates
    print(f"[ingest] {len(chunks)} chunks embedded and stored.")

if __name__ == "__main__":
    build_knowledge_base()

```

This is the offline plane, condensed from Modules 2–4: a `DirectoryLoader` ingests a whole folder of PDFs, a validation step drops failed extractions, the recursive splitter chunks them, metadata is enriched and — critically — **deterministic IDs** are assigned so re-ingesting an updated document overwrites its old chunks rather than duplicating them. Chunks are embedded with Gemini and stored in a *persistent* Chroma database. This script is run as a batch job whenever the documents change; the serving plane never runs it.

10.7 Component 2 — Hybrid Retrieval with Re-ranking

The retriever combines Module 5's hybrid search with Module 6's two-stage re-ranking.

```

# src/retriever.py - hybrid retrieval + cross-encoder re-ranking
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_chroma import Chroma
from langchain_community.retrievers import BM25Retriever
from langchain.retrievers import EnsembleRetriever, ContextualCompressionRetriever
from langchain.retrievers.document_compressors import CrossEncoderReranker
from langchain_community.cross_encoders import HuggingFaceCrossEncoder
from src import config

def build_retriever():
    """Hybrid (dense + BM25) retrieval, then cross-encoder re-ranking."""
    embeddings = GoogleGenerativeAIEmbeddings(model=config.EMBEDDING_MODEL)
    store = Chroma(collection_name=config.COLLECTION_NAME,
                    embedding_function=embeddings,
                    persist_directory=config.CHROMA_DIR)

    # Dense (semantic) retriever - fetch a BROAD candidate set
    dense = store.as_retriever(search_kwargs={"k": config.RETRIEVE_K})

    # Sparse (BM25 keyword) retriever - built from the stored chunks
    all_chunks = store.get() # pull stored documents
    from langchain_core.documents import Document
    docs = [Document(page_content=t, metadata=m)
              for t, m in zip(all_chunks["documents"], all_chunks["metadatas"])]
    sparse = BM25Retriever.from_documents(docs)
    sparse.k = config.RETRIEVE_K

    # Stage 1: HYBRID - fuse dense + sparse with Reciprocal Rank Fusion
    hybrid = EnsembleRetriever(retrievers=[dense, sparse], weights=[0.5, 0.5])

    # Stage 2: RE-RANK the broad candidate set down to the best few
    reranker = CrossEncoderReranker(
        model=HuggingFaceCrossEncoder(model_name=config.RERANKER_MODEL),
        top_n=config.RERANK_TOP_N)

    return ContextualCompressionRetriever(
        base_compressor=reranker, base_retriever=hybrid)

```

This is the two-stage retrieve-then-rerank pipeline. Stage 1 is hybrid retrieval — a dense Chroma retriever and a sparse BM25 retriever, fused by an `EnsembleRetriever` — fetching a deliberately *broad* candidate set of 20 chunks to maximise recall. Stage 2 is a cross-encoder re-ranker, wrapped in a `ContextualCompressionRetriever`, which scores those 20 candidates precisely and keeps the best 4. The result is a retriever that understands meaning *and* exact words, casts a wide net, *and* delivers a precisely-ordered final context — the production-grade retrieval backbone from Modules 5 and 6.

10.8 Component 3 — The Agentic RAG Core

The brain of the system is the self-correcting LangGraph workflow from Module 8.


```

# src/graph.py – the agentic RAG core (Module 8)
from typing import TypedDict, List
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.documents import Document
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver
from src.retriever import build_retriever
from src import config

llm = ChatGoogleGenerativeAI(model=config.LLM_MODEL, temperature=0)
retriever = build_retriever()

class RAGState(TypedDict):
    question: str          # current (possibly rewritten) query
    original: str           # the user's true question
    documents: List[Document]
    answer: str
    citations: List[str]
    retries: int

# --- LLM-powered helpers ---
grader = (ChatPromptTemplate.from_template(
    "Is this document relevant to the question? Reply only 'yes' or 'no'.\n"
    "Question: {q}\nDocument: {d}") | llm | StrOutputParser())

answerer = (ChatPromptTemplate.from_template(
    "You are an enterprise knowledge assistant. Answer the QUESTION using "
    "ONLY the CONTEXT. If the context does not contain the answer, say so "
    "honestly. Be accurate and concise.\n\nCONTEXT:\n{ctx}\n\n"
    "QUESTION: {q}") | llm | StrOutputParser())

rewriter = (ChatPromptTemplate.from_template(
    "Rewrite this question as a clearer search query. Return only the "
    "query.\n\nQuestion: {q}") | llm | StrOutputParser())

# --- Nodes ---
def retrieve(state: RAGState) -> dict:
    return {"documents": retriever.invoke(state["question"])}

def grade(state: RAGState) -> dict:
    relevant = [d for d in state["documents"]
                if "yes" in grader.invoke(
                    {"q": state["original"], "d": d.page_content}).lower()]
    return {"documents": relevant}

def rewrite(state: RAGState) -> dict:
    return {"question": rewriter.invoke({"q": state["original"]}).strip(),
            "retries": state["retries"] + 1}

def generate(state: RAGState) -> dict:
    ctx = "\n\n".join(d.page_content for d in state["documents"])
    answer = answerer.invoke({"ctx": ctx, "q": state["original"]})
    citations = sorted({d.metadata.get("file_name", "unknown")
                        for d in state["documents"]})
    return {"answer": answer, "citations": citations}

# --- Conditional routing (with loop guard) ---
def decide(state: RAGState) -> str:
    if state["documents"]:
        return "generate"
    if state["retries"] < config.MAX_RETRIES:
        return "rewrite"

```

```

return "generate"                # exhausted retries → answer honestly

# --- Assemble & compile the graph ---
def build_graph():
    g = StateGraph(RAGState)
    g.add_node("retrieve", retrieve)
    g.add_node("grade", grade)
    g.add_node("rewrite", rewrite)
    g.add_node("generate", generate)
    g.add_edge(START, "retrieve")
    g.add_edge("retrieve", "grade")
    g.add_conditional_edges("grade", decide,
                           {"generate": "generate", "rewrite": "rewrite"})
    g.add_edge("rewrite", "retrieve")    # self-correction loop
    g.add_edge("generate", END)
    return g.compile(checkpointer=MemorySaver())    # checkpointer = memory

rag_graph = build_graph()

```

This is the Corrective-RAG-style agentic graph from Module 8, now wired to the production retriever. It retrieves, **grades** its own results, and if nothing relevant was found it **rewrites the query and retries** — with a `MAX_RETRIES` loop guard so it can never loop forever. When it does answer, it builds **citations** from the chunks' `file_name` metadata, fulfilling the traceability promise of Module 1. The `MemorySaver` checkpointer gives the system conversation memory: requests sharing a `thread_id` form one continuous session.

10.9 Component 4 — The Evaluation Suite

Quality is measured, not assumed. `evaluate.py` applies Module 9's RAGAS suite to the assembled system.

```
# src/evaluate.py — RAGAS evaluation of the full system (Module 9)
import json
from ragas import evaluate, EvaluationDataset
from ragas.metrics import (Faithfulness, ResponseRelevancy,
                           LLMContextPrecisionWithReference, LLMContextRecall)
from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper
from langchain_google_genai import (ChatGoogleGenerativeAI,
                                    GoogleGenerativeAIEmbeddings)

from src.graph import rag_graph
from src import config

def run_evaluation(test_set_path="tests/test_set.json"):
    """Run the real system over a test set and score it with RAGAS."""
    test_set = json.load(open(test_set_path))

    samples = []
    for item in test_set:
        # Run the ACTUAL system to get its genuine answer + contexts
        state = rag_graph.invoke(
            {"question": item["question"], "original": item["question"],
             "documents": [], "retries": 0},
            config={"configurable": {"thread_id": f"eval-{item['id']}"}})
        samples.append({
            "user_input": item["question"],
            "response": state["answer"],
            "retrieved_contexts": [d.page_content for d in state["documents"]],
            "reference": item["ground_truth"],
        })

    judge_llm = LangchainLLMWrapper(
        ChatGoogleGenerativeAI(model=config.LLM_MODEL, temperature=0))
    judge_emb = LangchainEmbeddingsWrapper(
        GoogleGenerativeAIEmbeddings(model=config.EMBEDDING_MODEL))

    return evaluate(
        dataset=EvaluationDataset.from_list(samples),
        metrics=[Faithfulness(), ResponseRelevancy(),
                 LLMContextPrecisionWithReference(), LLMContextRecall()],
        llm=judge_llm, embeddings=judge_emb)
```

Crucially — exactly as Module 9 stressed — this script **runs the real system** to obtain each sample's `response` and `retrieved_contexts`; only the `question` and `ground_truth` come from the hand-authored `test_set.json`. The result is an honest, four-metric score of the *actual* deployed pipeline, which you re-run after every change.

10.10 Component 5 — Monitoring with LangSmith

A deployed agentic system makes many LLM calls, follows dynamic paths, and runs where you cannot watch it. **LangSmith** is the observability platform that records every step: each request becomes a *trace* showing every node the graph visited, every retrieval, every LLM call, with inputs, outputs, latency, and token counts.

The remarkable thing is how little code monitoring requires. LangChain and LangGraph emit traces *automatically* once a few environment variables are set:

```
# add to .env — monitoring switches on with no code changes
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=your_langsmith_api_key
LANGCHAIN_PROJECT=enterprise-rag-production
```

With these set, every `rag_graph.invoke()` call is traced to your LangSmith project with no change to the application code. In the LangSmith dashboard you can then see, for any request: the exact path the agent took (did it loop and retry?), what each retrieval returned, what each LLM call cost in tokens and time, and where errors or slow steps occurred. This is what makes a production agent *debuggable* — and Module 8 warned that an agent's dynamic path is undebuggable without it.

WHY THIS MATTERS

The difference between a hobby project and a production system is, to a large degree, observability. When a hobby project misbehaves, you re-run it and stare at it. When a production system misbehaves — for a user, at 3 a.m. — you cannot re-run it; you need a recorded trace of *what actually happened on that request*. LangSmith provides that. Monitoring is not optional polish; it is the only way to operate, debug, and improve a deployed agentic RAG system, and the only way to catch a quality regression before users do.

10.11 Component 6 — API Deployment with FastAPI

To make the system reachable, we wrap the agentic graph in a web API. **FastAPI** is the standard modern Python framework for this — fast, simple, and self-documenting.

```
# src/app.py — ONLINE: the FastAPI service
from fastapi import FastAPI
from pydantic import BaseModel
from src.graph import rag_graph

app = FastAPI(title="Enterprise Knowledge Assistant")

class ChatRequest(BaseModel):
    question: str
    session_id: str = "default"      # groups a conversation (memory)

class ChatResponse(BaseModel):
    answer: str
    citations: list[str]

@app.get("/health")                  # for load balancers & uptime checks
def health():
    return {"status": "ok"}

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest):
    """Answer one question, with conversation memory keyed on session_id."""
    result = rag_graph.invoke(
        {"question": request.question, "original": request.question,
         "documents": [], "retries": 0},
        config={"configurable": {"thread_id": request.session_id}},
    )
    return ChatResponse(answer=result["answer"],
                        citations=result["citations"])
```

Run it locally with an ASGI server:

```
uvicorn src.app:app --host 0.0.0.0 --port 8000
```

The design is deliberately thin. The API layer does almost nothing itself — it defines a typed request (`ChatRequest`, validated automatically by Pydantic) and a typed response, exposes a `/health` endpoint for load balancers, and a `/chat` endpoint that simply hands the question to the agentic graph and returns the result. **All intelligence stays in the graph; the API is just a doorway.** The `session_id` flows through as the graph's `thread_id`, so each user's conversation memory is kept separate. One subtlety worth noting: this graph object is *shared* across requests, so the API process is effectively stateless per request (state lives in the checkpointers keyed by `thread_id`) — which is exactly the property that lets us scale it horizontally in Section 10.14.

10.12 Component 7 — Docker Deployment

The API runs on your machine. To run it *anywhere* — reliably, identically — you containerise it with **Docker**. A container packages the application together with its exact Python version, dependencies, and environment, so it behaves the same on your laptop, a colleague's machine, and a cloud server. It is the definitive cure for "it works on my machine."

```
# Dockerfile – defines the container image
FROM python:3.11-slim

WORKDIR /app

# Install dependencies first (this layer is cached between builds)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy the application code and the pre-built knowledge base
COPY src/ ./src/
COPY chroma_db/ ./chroma_db/

EXPOSE 8000

# Start the API server
CMD ["uvicorn", "src.app:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# requirements.txt – pinned for reproducible builds
fastapi
uvicorn
langchain
langchain-community
langchain-google-genai
langchain-chroma
langgraph
faiss-cpu
rank-bm25
sentence-transformers
pypdf
ragas
python-dotenv
```

Build and run the container:

```
docker build -t enterprise-rag .
docker run -p 8000:8000 --env-file .env enterprise-rag
```

A few decisions in this Dockerfile reflect real practice. `python:3.11-slim` is a small base image — smaller images deploy faster. `requirements.txt` is copied and installed *before* the application code, so Docker's layer caching does not reinstall every dependency each time you change a line of code. The pre-built `chroma_db/` is copied in, so the container ships with its knowledge base ready. And the `.env` file is passed at *run* time with `--env-file`, never baked into the image — secrets must never live inside a container image.

BEST PRACTICE

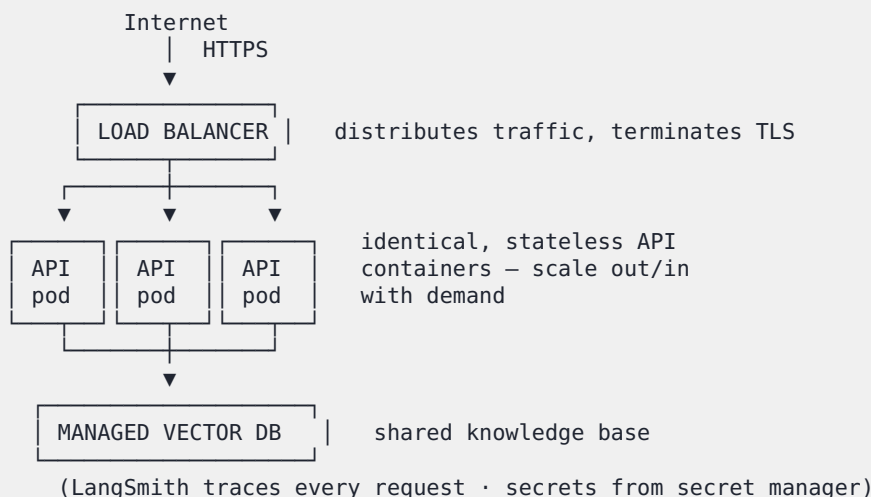
Keep the ingestion step *out* of the serving container's runtime. Build the knowledge base first (run `ingest.py` as a separate job), then either copy the resulting `chroma_db/` into the image as shown, or — better at scale — point the serving container at an external, managed vector database. A serving container should *start fast and start ready*: it loads an existing knowledge base, it never builds one. Mixing ingestion into startup makes the service slow to boot and couples two things that should scale independently.

10.13 Component 8 — Cloud Deployment

A container on your laptop is not yet a product. **Cloud deployment** puts the container on infrastructure that is always on, reachable from the internet, and able to grow with demand. The container image you built is portable, so the deployment target is a choice rather than a constraint. The common options:

- **Managed container services** (such as cloud "run a container" platforms) — you hand the platform your image; it runs it, gives it a public URL, handles HTTPS, and scales the number of instances up and down with traffic, often to zero when idle. This is the simplest path and the right default for most RAG APIs.
- **Container orchestration** (Kubernetes and similar) — for large systems needing fine-grained control over scaling, networking, and multiple coordinated services. More powerful, more operational overhead.
- **Platform-as-a-service** — a hosting platform that takes your code or image and runs it with minimal configuration. Fast to start, less control.

Whichever you choose, a production cloud deployment shares the same shape: the container runs behind a **load balancer** (which distributes traffic and terminates HTTPS), the platform runs **multiple instances** of the container for capacity and redundancy, **secrets** are injected from the platform's secret manager rather than a file, and the vector store is an **external managed database** (Module 4) rather than a file inside the container — so that many serving instances can share one knowledge base.



10.14 Production Scalability

A system that works for ten users may collapse for ten thousand. **Scalability** is designing so that growth is handled by adding resources, not by rewriting the system. The principles, several of which the architecture above already builds in:

- **Stateless serving instances.** Because per-conversation state lives in the checkpoint (keyed by `thread_id`), not in the API process's memory, any instance can handle any request. This is what makes **horizontal scaling** possible — to handle more load, you simply run more identical instances behind the load balancer.
- **Separate the planes.** Ingestion and serving scale independently. A surge of users needs more *serving* instances; a surge of new documents needs more *ingestion* capacity. Coupling them forces you to over-provision both.
- **A managed, scalable vector database.** The shared knowledge base must itself scale (Module 4's production architecture: sharding, replicas). A file-based store on one machine becomes the bottleneck.
- **Caching.** Many queries repeat. Caching embeddings and answers for repeated or similar queries cuts latency and cost dramatically. (The Production RAG section covers caching strategies in depth.)
- **Asynchronous and queued processing.** Long-running work (ingesting a large document set) should run on a background queue, not block a user request.
- **Autoscaling.** The platform should add instances when traffic rises and remove them when it falls — paying for capacity only when it is needed.
- **Rate limiting and graceful degradation.** Protect the system from overload and from runaway costs; when a dependency is slow, degrade gracefully rather than failing hard.

WHY THIS MATTERS

Scalability is not something you bolt on after launch — it is decided by the architecture you choose at the start. The two decisions that matter most were made early in this capstone: keeping the serving layer *stateless* (so it can be replicated freely) and keeping the vector store *external and managed* (so all replicas share one knowledge base). Get those two right and scaling is mostly a matter of turning up a number. Get them wrong — state trapped in the process, a knowledge base locked inside one container — and scaling means a painful rebuild. Architecture is the cheapest time to buy scalability.

10.15 Putting It Together: The Capstone Workflow

The full lifecycle of the deployed system:

1. **Configure** — fill in `.env` (Gemini and LangSmith keys) and `config.py`.

2. **Ingest** — place documents in `data/`, run `ingest.py` to build the persistent knowledge base. Re-run only when documents change.
3. **Evaluate** — run `evaluate.py` against `tests/test_set.json` to get baseline RAGAS scores. Tune (one variable at a time, Module 9) until quality is acceptable.
4. **Serve locally** — run the FastAPI app with `uvicorn`; test the `/chat` endpoint.
5. **Containerise** — build the Docker image; run and test the container.
6. **Deploy** — push the image to a cloud container service behind a load balancer, with secrets from a secret manager and an external managed vector database.
7. **Monitor** — watch LangSmith traces; re-run the evaluation suite on a schedule to catch regressions.
8. **Iterate** — improve a component, re-evaluate, re-deploy. The loop never stops; that is what operating a system means.

10.16 Common Mistakes

- **Tangling ingestion into serving.** Building the knowledge base inside a user request, or inside container startup, makes the service slow and unscalable. Keep the two planes separate.
- **Putting state in the API process.** State in process memory breaks horizontal scaling. Keep serving instances stateless; state lives in the checkpoint or an external store.
- **A file-based vector store in production.** A single-machine file store cannot be shared by many replicas and becomes the bottleneck. Use a managed, scalable vector database.
- **Baking secrets into the Docker image.** API keys in an image are a serious leak. Inject secrets at runtime from a secret manager.
- **No `/health` endpoint.** Load balancers and cloud platforms need a health check to route traffic and restart unhealthy instances.
- **Deploying without monitoring.** A production agent without LangSmith tracing is undebuggable when something goes wrong for a real user.
- **No loop guard on the agentic graph.** A retry loop without a `MAX_RETRIES` guard can run up an unbounded bill in production (Module 8).
- **Shipping without an evaluation baseline.** Deploying a system whose RAGAS scores you never measured means you cannot detect regressions or justify quality.

10.17 Best Practices

- **Separate the offline ingestion plane from the online serving plane**; let them scale independently.
- **Keep the serving layer thin and stateless** — the API is a doorway; the graph is the brain; state lives in the checkpoint.
- **Centralise configuration** in one file so model, chunk size, and parameters change in one place.
- **Use deterministic IDs** so re-ingestion overwrites rather than duplicates.
- **Use an external, managed vector database** for production so replicas share one knowledge base.
- **Containerise with pinned dependencies**; inject secrets at runtime, never bake them in.
- **Enable LangSmith tracing and run the RAGAS suite regularly** — observability and measurement are not optional.
- **Guard every agentic loop**, expose a health endpoint, and design for horizontal scaling and graceful degradation.

10.18 Improvements and Scaling Strategies

A capstone is a starting point. Realistic next steps to push it toward a mature product:

- **Streaming responses** — stream the answer token by token to the user for a faster perceived response.
- **Advanced patterns where measured to help** — add HyDE, RAG Fusion, or a Self-RAG faithfulness check (Module 7), each justified by an ablation evaluation (Module 9).
- **Multi-modal ingestion** — extend the pipeline to charts, diagrams, and scanned tables (Module 7), with Gemini as the multi-modal generator.
- **A web-search fallback tool** — give the agent a web-search tool so it can answer questions outside the knowledge base (true CRAG, Module 7).
- **Authentication and access control** — per-user authentication and metadata-based access control so users see only permitted documents (Modules 4, 6).
- **A caching layer** — cache embeddings and answers for repeated queries (Production RAG section).
- **CI/CD with an evaluation gate** — wire `evaluate.py` into the deployment pipeline so a change that drops RAGAS scores fails the build (Production RAG section).
- **Multi-agent expansion** — split into specialised agents under a supervisor for complex workflows (Module 8).

10.19 The Capstone as a Template: The Ten Projects

This capstone's architecture is general, and every project on the book's project list is a variation of it:

- **PDF Chatbot** — the capstone restricted to a single PDF.
- **Multi-PDF RAG System** — the capstone exactly as built, over many PDFs.
- **Website Chatbot** — swap the PDF loader for `WebBaseLoader` (Module 2).
- **YouTube Q&A Assistant** — swap the loader for a transcript loader (Module 2).
- **Enterprise Knowledge Assistant** — the capstone with authentication and access control added.
- **Research Assistant** — add a web-search tool and a plan-and-execute agent (Modules 7, 8).
- **Resume Analyzer** — the capstone over résumé documents, with a structured-output prompt.
- **Customer Support AI** — the capstone over help-centre articles, with escalation logic.
- **Multi-Agent RAG Application** — the capstone restructured as specialised agents under a supervisor (Module 8).
- **Graph-based RAG System** — replace the vector retriever with a knowledge-graph retriever (Module 7).

Build the capstone well and you have not built one project — you have built the *pattern* for all ten.

10.20 Chapter Summary

- The capstone integrates Modules 1–9 into one deployed system, the **Enterprise Knowledge Assistant**, and ships it.
- A production RAG system has two **separate planes**: an offline **ingestion plane** (build the knowledge base when documents change) and an online **serving plane** (always-on request handling). They share only the vector store.
- The system's components: **multi-document ingestion** with deterministic IDs (Modules 2–4); **hybrid retrieval + cross-encoder re-ranking** (Modules 5–6); a **self-correcting agentic LangGraph core** with conversation memory and a loop guard (Modules 7–8); a **RAGAS evaluation suite** run against the real system (Module 9); and **LangSmith monitoring**.
- A clean **project structure** mirrors the architecture: a central `config.py`, an offline `ingest.py`, shared `retriever.py` and `graph.py`, an `evaluate.py` quality instrument, and a thin `app.py` serving layer.

- **Deployment** proceeds in stages: wrap the graph in a thin **FastAPI** service (the API is a doorway; the graph is the brain); **containerise** with Docker (pinned dependencies, secrets injected at runtime, never baked in); deploy to a **cloud** container service behind a load balancer with an external managed vector database.
- **Monitoring** with LangSmith requires only environment variables — every request is then traced, making the dynamic agentic system debuggable in production.
- **Scalability** is an architectural decision made early: **stateless serving instances** (state in the checkpoint) enable horizontal scaling; **separating the planes** lets ingestion and serving scale independently; a **managed vector database, caching, autoscaling**, and **graceful degradation** complete the picture.
- The capstone's architecture is a **template** — every project on the book's list is a variation of it.

10.21 Capstone Assignment

This is the book's culminating project. Build and deploy the Enterprise Knowledge Assistant end to end.

1. **Set up.** Create the project structure of Section 10.5. Fill in `.env` and `config.py`. Place a real document set (at least 5 documents) in `data/`.
2. **Ingest.** Implement and run `ingest.py`. Confirm the persistent Chroma store is built with deterministic IDs and enriched metadata.
3. **Build retrieval.** Implement `retriever.py` — hybrid retrieval plus cross-encoder re-ranking. Test it directly on a few queries.
4. **Build the agentic core.** Implement `graph.py` — the self-correcting LangGraph workflow with a loop guard and citations. Test the in-scope and out-of-scope paths.
5. **Evaluate.** Write a `tests/test_set.json` of 15–20 questions with ground truth (include hard and out-of-scope ones). Implement and run `evaluate.py`. Record baseline RAGAS scores; diagnose and improve the weakest metric.
6. **Serve.** Implement `app.py`. Run it with `uvicorn` and test `/chat` and `/health`, including a multi-turn conversation that exercises memory.
7. **Containerise.** Write the `Dockerfile` and `requirements.txt`. Build the image and run the container; confirm the API works inside it.
8. **Monitor.** Enable LangSmith tracing. Send several requests and inspect the traces — confirm you can see the agent's path, including a retry loop.
9. **Deploy (stretch).** Deploy the container to a cloud container service. Confirm it is reachable over the internet with a public URL.
10. **Write it up.** Produce a short project report: the architecture, the baseline and improved RAGAS scores, one ablation result, the deployment steps, and a list of next

improvements. This report plus the deployed system is your capstone deliverable — and your portfolio centrepiece.

10.22 Interview Questions

BEGINNER

Q1. What are the two planes of a production RAG system?

The offline ingestion plane, which builds the knowledge base when documents change, and the online serving plane, which is always on and answers user requests. They run on different schedules and share only the vector store.

Q2. Why wrap a RAG system in an API?

So the outside world can reach it. An API (e.g. a FastAPI `/chat` endpoint) turns a script into a service that other applications, front-ends, and users can call over the network.

Q3. What problem does Docker solve?

It packages the application with its exact Python version, dependencies, and environment into a container that runs identically anywhere — eliminating "it works on my machine" and making deployment reproducible.

Q4. What is LangSmith used for?

Observability — it traces every request through the system, recording each step, LLM call, retrieval, latency, and token count, so a deployed agentic system can be monitored and debugged.

INTERMEDIATE

Q5. Why must ingestion and serving be kept separate?

They have different lifecycles and different scaling needs. Ingestion is an occasional batch job; serving is an always-on, low-latency service. Building the knowledge base inside a request — or inside container startup — would make the service slow and would couple two things that should scale independently.

Q6. Why should the API layer be thin and the agentic graph hold the logic?

Separation of concerns. A thin API (request validation, routing, the response) is easy to maintain and replace, and the intelligence — retrieval, grading, generation, looping — lives in one testable place: the graph. It also keeps the API process effectively stateless, which is what enables horizontal scaling.

Q7. How is conversation memory handled in the deployed system?

Through the LangGraph checkpointer keyed on a `thread_id`. The API maps each user's `session_id` to a `thread_id`; all requests on that thread share state, so the agent remembers earlier turns. State lives in the checkpointer, not in the API process — which keeps the API stateless.

Q8. Why should secrets never be baked into a Docker image?

A Docker image can be shared, pushed to registries, and inspected; a key baked into it is a serious leak. Secrets should be injected at runtime from the environment or a secret manager, keeping them out of the image entirely.

ADVANCED**Q9. What architectural decisions make a RAG serving system horizontally scalable?**

Two main ones. First, keep the serving instances stateless — per-conversation state lives in an external checkpointer or store keyed by `thread_id`, not in process memory — so any instance can serve any request and you scale by adding identical instances behind a load balancer. Second, use an external, managed, scalable vector database rather than a file-based store, so all instances share one knowledge base. Separating the ingestion and serving planes so they scale independently, plus caching and autoscaling, completes the picture.

Q10. Describe an end-to-end cloud deployment architecture for this system.

Traffic arrives over HTTPS at a load balancer that terminates TLS and distributes requests. Behind it run multiple identical, stateless API container instances, autoscaled with demand. The containers share an external managed vector database holding the knowledge base. Secrets are injected from the platform's secret manager, not from a file. Conversation state lives in a persistent checkpointer. Every request is traced to LangSmith, and a RAGAS evaluation suite runs on a schedule to catch quality regressions. Ingestion runs as a separate batch job that updates the shared vector database.

Q11. How would you build CI/CD for this RAG system, and what makes it different from CI/CD for ordinary software?

The pipeline builds the Docker image, runs standard tests, and then — the AI-specific part — runs the RAGAS evaluation suite against a fixed test set, treating the metric scores as a quality gate: if faithfulness, relevancy, precision, or recall drop below an agreed threshold, the build fails and the change is not deployed. It differs from ordinary CI/CD because correctness is not a binary pass/fail of deterministic tests but a measured quality score over open-ended outputs, so the gate is a metric threshold rather than an assertion. This catches quality regressions before they reach users.

SCENARIO-BASED

Q12. Your RAG prototype works perfectly in a notebook. Your manager asks you to "put it in production." Walk through what that actually involves.

Separate the code into an offline ingestion plane and an online serving plane. Restructure into a clean project with central configuration. Build the knowledge base in a persistent, ideally managed, vector store. Wrap the system in a thin API (FastAPI) with a chat endpoint and a health endpoint, keeping the serving layer stateless so it can scale. Containerise it with Docker, with pinned dependencies and secrets injected at runtime. Deploy the container to a cloud service behind a load balancer, with multiple autoscaling instances and an external vector database. Add LangSmith monitoring so the system is observable, and a RAGAS evaluation suite so quality is measured and regressions are caught. Add loop guards, rate limiting, and graceful degradation. In short: a notebook becomes a product through separation of planes, a thin stateless API, containerisation, scalable cloud infrastructure, observability, and continuous evaluation.

Q13. A week after deploying, users report the assistant has become slower and occasionally gives worse answers, though no code was changed. How do you investigate?

Start with the observability and measurement already built in. Check LangSmith traces for recent requests: are they slower at a particular step — retrieval, re-ranking, an LLM call — or is the agent looping and retrying more often? That localises a latency regression. For quality, re-run the RAGAS evaluation suite and compare the scores to the recorded baseline: if context recall or precision dropped, retrieval has degraded; if faithfulness dropped, generation has. Likely root causes with no code change include the knowledge base having grown or drifted after a re-ingestion, an upstream model or dependency change, or infrastructure load. The fix follows the diagnosis — but the key point is that because the system was deployed with tracing and a fixed evaluation baseline, the investigation is a measured procedure rather than guesswork.

END OF MODULE 10 — AND THE CORE CURRICULUM

You have done the full journey: from "what is RAG?" to a deployed, monitored, self-correcting agentic RAG system serving real users. You can ingest, retrieve, re-rank, reason agentially, evaluate, deploy, and scale. One section remains — the dedicated **Production RAG** chapter — which goes deep on the operational craft that keeps a deployed system fast, cheap, secure, and reliable: pipeline and latency optimisation, caching, cost and token optimisation, logging and tracing, security, compliance, guardrails, prompt-injection prevention, and CI/CD for AI systems.